



# Шлях до кар'єри Python Fullstack розробника

## МОДУЛЬ 1. PYTHON CORE

Рівень 25 Git: доступи, порядок і командна  
робота



# План на сьогодні

- 1 **.gitignore** — що не має потрапити в репозиторій  
синтаксис, шаблони, `git rm --cached`
- 2 **Доступи до GitHub** — SSH-ключі та токени  
налаштовуємо з нуля, наживо
- 3 **«Я помилився»** — amend, reset, revert, stash  
три режими reset: soft / mixed / hard

## Формат

Мінімум теорії — і одразу термінал. Усе, що на слайдах, робимо наживо.

- 4 **Pull Request і code review**  
як код потрапляє в main у реальній команді
- 5 **CI** — автоматична перевірка коду  
GitHub Actions на живому Python-проєкті

# .gitignore — навіщо він потрібен

Git за замовчуванням хоче відстежувати **усе**. А треба — не все.

- **Сміття оточення:** `venv/`, `__pycache__/`, `.idea/` — це не код, це стан *ТВОЄЇ* машини
- **Згенеровані файли:** `*.pyc`, білди, логи — їх завжди можна створити заново
- **Секрети:** `.env` з паролями, токенами, ключами API
- **Важкі файли:** бази даних, медіа, датасети — репозиторій роздувається назавжди

## Реальний кейс

Розробник закомітив `.env` з ключем AWS. Бот знайшов його на GitHub за 4 хвилини й підняв ферму для майнінгу.

Рахунок — кілька тисяч доларів за ніч.

## І прозаїчніше

Ти комітиш `.idea/` — колега робить `pull` і йому ламає налаштування IDE. Далі — конфлікт у файлі, який взагалі не код.

# Синтаксис .gitignore

| Патерн         | Що означає                      | Приклад: що потрапить під правило                      |
|----------------|---------------------------------|--------------------------------------------------------|
| *.log          | * — будь-яка кількість символів | app.log, logs/debug.log — усі .log на будь-якому рівні |
| temp?.txt      | ? — рівно один символ           | temp1.txt, tempA.txt, але <b>не</b> temp10.txt         |
| *.[oa]         | [ ... ] — один символ зі списку | main.o, lib.a                                          |
| venv/          | / в кінці — тільки тека         | тека venv будь-де; файл із такою назвою — ні           |
| /config.py     | / на початку — тільки корінь    | config.py у корені, але <b>не</b> app/config.py        |
| **/temp        | ** — будь-яка вкладеність       | temp, a/temp, a/b/c/temp                               |
| !important.log | ! — виняток із правила вище     | ігноруємо всі *.log, але цей — залишаємо               |
| # коментар     | # — рядок-коментар              | Git його не читає                                      |

## Порядок має значення

Правила читаються згори вниз. `!important.log` спрацює, тільки якщо стоїть **після** `*.log`. І виняток не витягне файл із теки, яка вже цілком ігнорується.

# Не пиши з нуля — бери шаблон

## РОБОЧИЙ МІНІМУМ ДЛЯ PYTHON

```
# Оточення та кеш
venv/
.venv/
__pycache__/
*.py[cod]

# IDE
.idea/
.vscode/

# Секрети
.env
*.pem

# Дані та артефакти
*.sqlite3
.pytest_cache/
.ruff_cache/
```

- **Прямо в GitHub:** при створенні репозиторію є список **Add .gitignore** — обери **Python**
- **Каталог шаблонів:** [github.com/github/gitignore](https://github.com/github/gitignore) — офіційна колекція під усі мови
- **Генератор:** [toptal.com/developers/gitignore](https://toptal.com/developers/gitignore) вводиш `python`, `pycharm`, `venv` → отримуєш готовий файл

### Правило

`.gitignore` створюємо **першим**, ще до першого `git add`. Тоді нічого зайвого просто не встигне потрапити в історію.

# А якщо файл **вже** в репозиторії?

Головне непорозуміння: `.gitignore` не діє на файли, які Git уже відстежує.

```
# .env вже закомічений — додали в .gitignore,  
# а він усе одно у змінах. Прибираємо з індексу:  
$ git rm --cached .env  
rm '.env'  
  
# цілу теку — рекурсивно:  
$ git rm -r --cached .idea/  
  
# перевіряємо і комітимо  
$ git status  
$ git commit -m "chore: stop tracking .env"
```

`--cached` — **це важливо**

Без нього `git rm` видалить файл **і з диска**. З ним — файл лишається у тебе, Git просто перестає його бачити.

## Але секрет уже скомпрометований

`git rm --cached` прибирає файл із **майбутніх** комітів. У старих комітах він лишається — і його видно через `git log` будь-кому, хто має репозиторій.

## ЩО РОБИТИ НАСПРАВДІ

- 1 **Відкликати** ключ / пароль і випустити новий
- 2 Прибрати файл з індексу й додати в `.gitignore`
- 3 Тільки потім — чистити історію ( `git filter-repo` ), якщо це взагалі потрібно

## ПРАКТИКА • 1

# Наводимо порядок у репозиторії

Створюємо проєкт з `venv` і `.env`, дивимось, що Git хоче їх забрати, додаємо `.gitignore` з шаблону — і рятуємо вже закомічений секрет через `git rm --cached`.

# Чому логін і пароль більше не працюють

- **13 серпня 2021** — GitHub повністю вимкнув автентифікацію паролем для Git-операцій
- Причина: пароль дає доступ **до всього акаунта** — усі репозиторії, налаштування, білінг
- Пароль не має терміну дії, його не можна обмежити одним проєктом і важко відкликати «частково»
- Тому GitHub перейшов на **токени** й **ключі**: обмежені за правами, за терміном і за областю дії

## Знайома помилка?

```
remote: Support for password authentication  
was removed on August 13, 2021.
```

Саме її бачить кожен, хто вперше робить `git push`.

## КЛЮЧОВА ІДЕЯ

### Доступ ≠ акаунт

Ти більше не «входиш» у GitHub з терміналу. Ти видаєш **конкретному пристрою** або **конкретному скрипту** обмежений ключ, який у будь-який момент можна відкликати — не змінюючи пароль і не ламаючи решту.

### Наслідок для нас

Поки доступ не налаштований, `git push` просто не працює. Тому це — обов'язковий крок, а не «додаткова опція».



# Три способи автентифікації

| Спосіб                                     | Як виглядає                                         | Коли брати                                                                  | Мінус                                                           |
|--------------------------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>SSH-ключ</b><br><b>рекомендовано</b>    | пара файлів на диску<br>id_ed25519 + .pub           | Твій власний комп'ютер, щоденна робота.<br>Налаштував один раз — і забув    | Треба<br>налаштувати; на<br>чужій машині так<br>робити не можна |
| <b>Personal Access<br/>Token<br/>(PAT)</b> | рядок github_pat_...<br>вводиться замість<br>пароля | CI/CD, скрипти, автоматизація, корпоративні<br>мережі, де SSH-порт закритий | Має термін дії; це<br>секрет, який<br>легко злити               |
| <b>OAuth через IDE</b>                     | кнопка «Log in with<br>GitHub» у PyCharm            | Швидкий старт, робота з GitHub прямо з IDE                                  | Працює тільки<br>всередині IDE; у<br>терміналі — ні             |

## Сьогодні робимо перші два

**SSH** — як основний спосіб для своєї машини. **PAT** — щоб розуміти, що вводити, коли SSH недоступний (і саме на ньому далі працюватиме наш CI).

# Як працює SSH-ключ

Замок і ключ. Замок можна віддавати всім, ключ — нікому.

ПУБЛІЧНИЙ · .PUB

**Замок** 

`id_ed25519.pub`

Завантажуєш на GitHub. Можеш надіслати в чат, показати на екрані, покласти на 10 серверів — нічого не станеться.

ПРИВАТНИЙ · БЕЗ РОЗШИРЕННЯ

**Ключ** 

`id_ed25519`

Лишається **тільки** на твоєму комп'ютері. Ніколи не комітиться, не надсилається і не показується на демо.

ПЕРЕВІРКА

**Рукоостискання**

GitHub надсилає випадкове завдання, яке можна розв'язати лише приватним ключем. Ключ при цьому **не передається** по мережі — передається тільки доказ, що він у тебе є.

## Чому саме ed25519, а не rsa

Коротший, швидший і криптографічно сильніший за `rsa 2048`. У всіх сучасних інструкціях GitHub — саме він. `rsa` побачиш у старих туторіалах.

# SSH за чотири кроки

```
# 1. Генеруємо пару ключів
$ ssh-keygen -t ed25519 -C "pochta@example.com"

Enter file... (/c/Users/user/.ssh/id_ed25519): ← Enter
Enter passphrase: ← пароль на ключ (можна порожній)

# 2. Дивимось ПУБЛІЧНИЙ ключ і копіюємо
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1... pochta@example.com

# 4. Перевіряємо зв'язок
$ ssh -T git@github.com
Hi username! You've successfully authenticated...
```

## 3. ДОДАЄМО ЗАМОК НА GITHUB

- 1 Аватар → **Settings**
- 2 **SSH and GPG keys** → **New SSH key**
- 3 Title — назва **пристрою** («Домашній ПК»), щоб потім знати, що відкликати
- 4 Key — вставляємо вміст `.pub` цілком, одним рядком

### Найчастіша помилка

Скопіювати `id_ed25519` замість `id_ed25519.pub`. Якщо в тексті є `PRIVATE KEY` — стоп, це не той файл.

# ssh-agent: щоб не вводити passphrase щоразу

```
# Git Bash: запускаємо агента в цій сесії
$ eval "$(ssh-agent -s)"
Agent pid 1234

# Додаємо ключ — passphrase питає один раз
$ ssh-add ~/.ssh/id_ed25519
Identity added: /c/Users/user/.ssh/id_ed25519

$ ssh-add -l    # що зараз в агенті
$ ssh-add -D    # прибрати всі ключі
```

## Нюанс Git Bash

`eval "$(ssh-agent -s)"` живе **лише до закриття вікна**. Нове вікно — знову вводити passphrase.

## ЩОБ ПРАЦЮВАЛО ПОСТІЙНО (WINDOWS)

Служба **OpenSSH Authentication Agent**: вмикаємо один раз — і ключ підхоплюється в будь-якому терміналі.

```
# PowerShell від адміністратора
> Set-Service ssh-agent -StartupType
Automatic
> Start-Service ssh-agent
# далі — у звичайному Git Bash
$ ssh-add ~/.ssh/id_ed25519
```

## Або простіше

Залишити passphrase порожнім — тоді агент не потрібен узагалі. Але кожен, хто дістанеться твого диска, дістанеться й репозиторіїв.

# Репозиторій уже клонований по HTTPS

Ключ згенерували, а `push` усе одно просить пароль? Бо remote досі HTTPS.

```
# Дивимось, куди зараз дивиться origin
$ git remote -v
origin https://github.com/user/repo.git (fetch)
origin https://github.com/user/repo.git (push)

# Перемикаємо на SSH
$ git remote set-url origin git@github.com:user/repo.git

# Перевіряємо
$ git remote -v
origin git@github.com:user/repo.git (fetch)

$ git push
Everything up-to-date ← пароля не спитало
```

## ДВА ФОРМАТИ ОДНОГО РЕПОЗИТОРІЮ

| HTTPS                                | SSH                              |
|--------------------------------------|----------------------------------|
| https://github.com/<br>user/repo.git | git@github.com:<br>user/repo.git |
| потребує токен                       | потребує ключ                    |

### Де взяти правильне посилання

На сторінці репозиторію — зелена кнопка **Code**, вкладка **SSH**. Не вигадуй адресу руками.

### Перший раз питає

Are you sure you want to continue connecting? → пишемо `yes`. Це Git запам'ятовує відбиток сервера GitHub.

# Personal Access Token

Settings → Developer settings → Personal access tokens

## БЕРИ ЦЕЙ

### Fine-grained token

Обираєш **конкретні репозиторії** й права на них.

- Repository access → **Only select repositories**
- Permissions → **Contents: Read and write**
- **Metadata: Read-only** — ставиться саме
- Expiration — 30–90 днів

```
github_pat_11ABCDE...
```

## ПОБАЧИШ У СТАРИХ ІНСТРУКЦІЯХ

### Classic token

Права — **на весь акаунт одразу**, усі репозиторії без винятку.

- scope `repo` — повний доступ до коду
- scope `workflow` — правити CI

```
ghp_AbCdEf123...
```

## ГОЛОВНЕ

### Токен = пароль

GitHub покаже його **рівно один раз**. Не зберіг — створюй новий.

Не комітити, не вставляти в код, не показувати на екрані.  
Витік — одразу **Revoke**.

### Куди його вводити

При `git push` по HTTPS: **Username** — твій логін, **Password** — вставляєш токен. Windows запам'ятає його у **Credential Manager** і більше не питатиме.

# Що і коли брати

## SSH

### Твоя машина

- Домашній або робочий ПК, який ти контролюєш
- Налаштував раз — працює роками, нічого не «протухає»
- Ключ не вводиться руками → не потрапить у скріншот чи історію команд
- Загубив ноутбук — видалив один ключ на GitHub, решта пристроїв працює

## PAT

### Все інше

- **CI/CD і скрипти** — сервер не має «свого» ключа, зате може отримати токен
- **Корпоративна мережа**, де закритий порт 22 (SSH просто не з'єднається)
- **Тимчасовий доступ** — видав на 7 днів, і він сам помре
- Коли потрібні дрібні права: тільки читання, тільки один репозиторій

### Практичне правило

На своєму комп'ютері — **SSH**. У чужому чи автоматизованому середовищі — **токен з мінімальними правами й коротким терміном**.

# Налаштовуємо доступ з нуля

Термінал, у якому GitHub мене не знає: жодного ключа, жодного збереженого токена. Генеруємо `ed25519`, вантажимо публічний ключ, перемикаємо `origin` на SSH — і робимо перший `push`. Потім те саме через токен.



ПАУЗА

# Перерва 10 хвилин

Питання, які лишились по доступах — пишiть у чат, після перерви розберемо перед наступним блоком.

# Три зони, які треба тримати в голові

Без цієї картинки `reset` здається магією. З нею — стає очевидним.

## 1 • Working Directory

Файли, які ти зараз бачиш у редакторі. Тут живуть незбережені правки.

**у `git status`:** червоне,  
«Changes not staged»

## 2 • Staging Area (Index)

Те, що ти відібрав у наступний коміт через `git add`.

**у `git status`:** зелене,  
«Changes to be committed»

## 3 • HEAD (репозиторій)

Історія комітів. Те, що вже зафіксовано назавжди.

**у `git log`:** список комітів  
з хешами

файл змінено

Working Dir



**`git add`**

кладемо в Index



**`git commit`**

фіксуємо в HEAD



**`git push`**

відправляємо на GitHub

## Далі — рух у зворотний бік

`amend`, `reset`, `revert` і `stash` відрізняються рівно одним: **скільки з цих трьох зон вони чіпають.**

# git commit --amend

Найлегший випадок: щойно закомітив — і одразу побачив помилку.

```
# Помилка в тексті коміту
$ git commit --amend -m "feat: add user login"

# Забув додати файл у той самий коміт
$ git add forgotten_file.py
$ git commit --amend --no-edit
#           --no-edit = не чіпати текст коміту

$ git log --oneline
a1b2c3d feat: add user login ← новий хеш!
```

## Що насправді відбувається

Git не «редагує» коміт — комміти незмінні. Він створює **новий** коміт замість старого. Тому хеш змінюється.

## Тільки до push

Якщо коміт уже на GitHub, `amend` переписує історію, і Git відмовиться пушити:

**! [rejected] main → main (non-fast-forward)**

Далі спокуса зробити `push --force` — і затерти роботу колег.

## Золоте правило

**Локальна історія — твоя, можеш переписувати.**

**Спільна історія — недоторканна.**

Уже запушив? Виправляй новим комітом або через `revert`.

# git reset: три режими

Усі три відкочують HEAD назад. Різниця — що вони роблять зі змінами.

| Команда                                                                      | HEAD (історія) | Staging   | Робочі файли | Коли це те, що треба                                                           |
|------------------------------------------------------------------------------|----------------|-----------|--------------|--------------------------------------------------------------------------------|
| <code>git reset --soft HEAD~1</code>                                         | назад          | збережено | збережено    | Розбити один великий коміт на кілька; переробити повідомлення разом зі складом |
| <code>git reset HEAD~1</code><br>(= <code>--mixed</code> , за замовчуванням) | назад          | очищено   | збережено    | Закомітив не ті файли — хочу перебрати, що саме додавати                       |
| <code>git reset --hard HEAD~1</code>                                         | назад          | очищено   | ЗНИЩЕНО      | Експеримент повністю провалився — хочу чистий стан, як був                     |

## Прибрати файл зі staging

`git reset file.py` — без хеша й без `--hard`. Це просто «скасувати `git add`», файл не постраждає.

## --hard — єдина справді небезпечна

Незакомічені зміни після неї **не відновити нічим**. Git навіть не спитає підтвердження.

# reset vs revert

## RESET — СТИРАЄ

### Ніби коміта й не було

Історія переписується. Підходить, поки коміти живуть **тільки в тебе на диску**.

до: A — B — C

після: A — B

## REVERT — ВИПРАВЛЯЄ

### Новий коміт, що скасовує старий

Історія лишається цілою — видно, що було зламано і коли це відкотили. Безпечно для спільної гілки.

до: A — B — C

після: A — B — C — C'

```
# Скасувати конкретний коміт безпечно
$ git revert a1b2c3d
[main d4e5f6g] Revert "feat: broken payment"
```

```
# Скасувати merge-коміт
$ git revert -m 1 <merge-hash>
```

### Правило одним рядком

Коміт уже на GitHub і його бачили інші → **тільки revert**.

### git reflog — рятівний круг

Зробив `--hard` не туди? Git місяцями пам'ятає, де був HEAD:

```
$ git reflog
$ git reset --hard HEAD@{2}
```

Рятує **коміти**. Не рятує зміни, які не встигли стати комітом.

# git stash — «поклади поки що в шухляду»

Пишеш фічу, вона наполовину готова — і тут прилітає терміновий баг на проді.

```
# Ховаємо незавершені зміни (-u = і нові файли)
$ git stash -u -m "половина форми логіну"

$ git status
nothing to commit, working tree clean

# Спокійно фіксимо терміновий баг
$ git switch -c hotfix/payment

# Повертаємось і дістаємо назад
$ git switch feature/login
$ git stash list
stash@{0}: On feature/login: половина форми...
$ git stash pop
```

| Команда                           | Що робить                                               |
|-----------------------------------|---------------------------------------------------------|
| git stash                         | Сховати зміни, повернути чисту робочу теку              |
| git stash -u                      | Те саме + нові файли, яких Git ще не бачив              |
| git stash list                    | Показати всі схованки                                   |
| git stash pop                     | Дістати останню і <b>видалити</b> зі списку             |
| git stash apply<br>git stash drop | Дістати, але <b>залишити</b> копію<br>Викинути схованку |

## Не перетворюй stash на смітник

Схованки не синхронізуються з GitHub, і через тиждень ти не згадаєш, що лежить у `stash@{4}`. Для чогось довшого за годину — краще гілка.

## ПРАКТИКА • 3

# Ламаємо і чинимо

Робимо «поганий» коміт і виправляємо його чотирма способами:

`amend`, три режими `reset`, `revert` для вже запущеного — і `reflog`, щоб повернути те, що «втратили назавжди».

# GitHub Flow: як код потрапляє в main

Так працює переважна більшість команд і майже весь open source.



- `main` **завжди робочий**. Будь-якої миті з нього можна зробити реліз
- **Ніхто не пушить у main напряму**. Навіть тімлід, навіть «на одну літеру»
- **Гілка = одна задача**. Не «мої зміни за тиждень», а «додати логін»

## Назви гілок — конвенція, як і в комітів

feature/user-profile  
fix/login-error-500  
docs/update-readme  
hotfix/payment-crash

## Branch protection

У Settings репозиторію можна **заборонити** push у main технічно: тільки через PR, тільки із зеленим CI, тільки після approve.



# Анатомія Pull Request

## З ЧОГО СКЛАДАЄТЬСЯ

- **base** ← **compare** — «влити `feature/login` у `main`».  
Найчастіша помилка новачка — переплутати напрямки
- **Conversation** — обговорення й історія подій
- **Commits** — усі коміти гілки
- **Files changed** — той самий `git diff`, тільки зручний: тут і лишають коментарі
- **Checks** — результати CI: ✓ passed ✗ failed

## Draft Pull Request

Відкривай PR **одразу**, ще на початку роботи, у статусі Draft. CI ганяється з першого коміту, а команда бачить, що ти робиш.

## ОПИС, ЯКИЙ НЕ ДРАТУЄ РЕВ'ЮЕРА

```
## Що зроблено
Додав ендпоінт /login і перевірку пароля.

## Як перевірити
1. pytest tests/test_auth.py
2. POST /login з валідними даними → 200

## Пов'язані задачі
Fixes #14 ← закрив issue #14 при merge
```

## ТРИ СПОСОБИ ЗЛИТИ

|                  |                                                             |
|------------------|-------------------------------------------------------------|
| Merge commit     | усі коміти + коміт злиття                                   |
| Squash and merge | уся гілка → <b>один</b> коміт у main<br><b>часто дефолт</b> |
| Rebase and merge | коміти лягають у main без злиття                            |

# Code review

Не екзамен і не критика особистості. Це друга пара очей на код, який піде в продакшн.

## ТРИ ВЕРДИКТИ РЕВ'ЮЕРА

### COMMENT

Питання без вироку — «а що буде, якщо список порожній?»

### APPROVE

Можна зливати. Дрібні побажання лишають необов'язковими ( nit: )

### REQUEST CHANGES

Є що виправити до merge. Не образа — нормальна частина процесу

## НА ЩО ДИВИТИСЬ

- **Чи працює** — логіка, граничні випадки, помилки
- **Чи зрозуміло** — назви й структура; чи розбереться інша людина за пів року
- **Чи безпечно** — секрети, SQL-ін'єкції, дані користувача
- **Чи є тести** на нову поведінку

### Suggestion — коментар, який комітиться

Рев'юер пише готову правку в блоці suggestion, автор тисне **Commit suggestion** — і вона вже в гілці.

### Стиль коду не обговорюють люди

Відступи, лапки, довжину рядка перевіряє **лінтер у CI**, а не колега в коментарях.

# CI — робот, який перевіряє код за тебе

Continuous Integration: на кожен push і кожен PR автоматично ганяються тести й лінтер.

- **Ловить те, що «у мене працює»** — забутий імпорт, залежність, якої немає в `requirements.txt`, тест, зламаний сусідньою правкою
- **Знімає суперечки про стиль** — `ruff` сказав, що не так, і це не чиясь думка
- **Дає видимий статус у PR** — червоний хрестик видно раніше, ніж рев'юер відкриє код
- **Це вже не «додатково»** — на будь-якій вакансії Junior+ CI є за замовчуванням

## GITHUB ACTIONS

### Вбудовано в GitHub

Нічого не треба піднімати й налаштовувати окремо. Кладеш YAML-файл у теку `.github/workflows/` — і воно вже працює.

Для публічних репозиторіїв — **безкоштовно**.

### Словник

**workflow** — файл-сценарій · **job** — окреме завдання на чистій віртуалці · **step** — один крок · **action** — готовий блок, який хтось уже написав

# Розбираємо `.github/workflows/ci.yml`

```
name: CI

on:
    # коли запускати
    push:
        branches: [main]
    pull_request:
        # на кожен PR

jobs:
    test:
        runs-on: ubuntu-latest # чиста віртуалка
        steps:
            - uses: actions/checkout@v4
            - uses: actions/setup-python@v5
              with:
                python-version: "3.12"
            - run: pip install -r requirements.txt
            - run: ruff check .
            - run: pytest -q
```

- `on:` — тригер. `pull_request` без параметрів = будь-який PR
- `runs-on:` — GitHub піднімає **чисту** Ubuntu. Саме тому CI і ловить «а в мене ж працювало»
- `uses:` — готовий action: `checkout` клонує код, `setup-python` ставить Python
- `run:` — звичайна команда в терміналі тієї віртуалки

## Найчастіша причина червоного CI

Немає `requirements.txt`. Робот бачить рівно те, що **закомічено** — не пакети, встановлені локально в тебе, і не файли з `.gitignore`.

Кроки йдуть згори вниз і зупиняються на першій помилці.

# Повний цикл на живому проєкті



Заодно перевіримо, що `venv` і `.env` у PR не потрапили — бо `.gitignore` ми написали ще на початку заняття.

# Підсумок

## ЧЕКЛИСТ ДЛЯ БУДЬ-ЯКОГО НОВОГО ПРОЄКТУ

- 1 `.gitignore` — до першого коміту
- 2 SSH-ключ налаштований, `origin` через `git@`
- 3 Секрети — тільки в `.env`, у репозиторії — `.env.example`
- 4 Робота — в гілках, у `main` — через PR
- 5 `ci.yml` з тестами й лінтером із першого дня

### Дві команди, з якими обережно

`git reset --hard` і `git push --force`. Перша знищує твою роботу, друга — чужу.

## КУДИ РУХАТИСЬ ДАЛІ

- **Pro Git** українською — `git-scm.com/book/uk/v2`
- **Learn Git Branching** — `learngitbranching.js.org`  
інтерактивний тренажер гілок просто в браузері
- **Git Howto** — `githowto.com/uk`
- **Oh Shit, Git!?!** — `ohshitgit.com`  
рецепти на випадок «я все зламав»

### Що варто зробити цього тижня

Взяти будь-який свій навчальний проєкт, залити на GitHub уже по SSH, додати `.gitignore` і `ci.yml` — і зробити в ньому один PR самому собі.

# Дякую за увагу!

## МОДУЛЬ 1. PYTHON CORE

Рівень 25 Git: доступи, порядок і командна робота

Питання — у чат

